130k Lines of Formal Topology in Two Weeks: Simple and Cheap Autoformalization for Everyone?

Josef Urban
AI4REASON and U. of Gothenburg/Chalmers U.

January 29, 2026, Mathematical Foundations seminar, U. of Bath

One-slide executive summary

- Goal: autoformalize a large chunk of *general topology* (Munkres) into a proof-checked library.
- Method: long-running **LLM** $\leftrightarrow$ **proof-checker feedback loop** + minimal glue/scripts.
- Outcome (paper snapshot): $\sim$160k lines by Jan 4, 2026; $\sim$130k lines produced Dec 22–Jan 4 for $\sim$ \$100 model-usage estimate.
- Outcome (latest): backup `bck5016` indicates substantially larger codebase and proof coverage (details later).
- Takeaway: "garage-hacker" autoformalization at surprising scale; key bottlenecks are workflow, context limits, and dependency hygiene.

Problem setting and why topology?

- Target corpus: Munkres, *Topology* (7 chapters, 39 sections, 241 pages).
- Topology is a stress test for:
  - heavy definitions + quantifiers + set reasoning,
  - long dependency chains (compactness/connectedness/separation axioms),
  - occasional analytic prerequisites (e.g., Tietze proof path).
- Formalization is not "solving"; it is **explicit proof object construction** with zero "it is obvious" moves.

High-level architecture: the loop

LLM coding agent
$\downarrow$ (edits .mg / adds defs + proofs)
Proof checker (Megalodon)
$\downarrow$ (errors/warnings or silence=success)
Return diagnostics to LLM
$\downarrow$ (iterate)
Backups + progress tracking

- "Silent output = success" makes compilation a crisp reward signal.
- The loop runs multiday; user intervenes mainly for steering/guardrails.

Core components (paper snapshot)

- LLM side: ChatGPT Pro subscription; model preference in the paper: GPT-5.2 w/ high reasoning; also experiments with Claude Code.
- Checker: Chad Brown's **Megalodon** (higher-order set theory), with mgwiki toolchain for HTML-ized browsing.
- Library base: Brown's set theory + surreal numbers (incl. reals), with many proofs replaced by `admit` for brevity/learnability.
- Ops: sandboxing via **bubblewrap** (bwrap), long-running session in **tmux**, periodic backups and change logs.

Why Megalodon (and why it matters)

- Higher-order set theory style: close to informal math vocabulary; classical reasoning available.
- Fast checking early; scales to tens of seconds on large files (paper reports ∼32s on a notebook for a large snapshot).
- mgwiki HTMLization provides a **browsable proof artifact**: clickable theorem graph feel for humans.
- Notably: the experiment is *not* tied to a mainstream assistant/library (Lean/Coq/Isabelle); suggests tool-agnostic future, training on the library unnecessary?

The "one prompt" trick: CLAUDE.md as policy

- Main loop prompt (repeated 1000–2000 times in paper snapshot):

  "Read the file `CLAUDE.md`. Treat it as authoritative work instructions... work as long as possible..."

- `CLAUDE.md` encodes:
  - **edit boundaries** (only topology section),
  - **no destructive reverts** (salvage rule; prevent loss),
  - **style constraints** (comment syntax, associativity gotchas),
  - **progress monitoring** and dependency tracking hooks.
- Think of it as a **micro-governance layer** for an autonomous coding agent.

Prompting automation: “babysitting” removed

- A small script watches tmux output and re-injects the prompt after inactivity.
- Practical effect:
  - converts interactive agent into a quasi-daemon,
  - pushes work toward “always-on” compilation-driven iteration,
  - makes the project mostly limited by model quotas + context management.
- For replication: the automation is arguably more important than model choice once basic competence is reached.

Quantitative progress: paper snapshot vs latest

Paper snapshot (Jan 4, 2026):

- ∼160k lines total; ∼130k lines produced Dec 22–Jan 4.
- Over 1.5k lemmas/theorems claimed at that stage.

Latest stats (January 29):

- `wc bck5016`: **251,576** lines; **1,830,306** words; **11,520,364** bytes.
- **2217** stated theorems/lemmas, of which **2124** are proved without recursive admits.

What counts as "proved" here?

- Megalodon supports `admit`; additionally, a theorem may be **recursively admitted** if it depends on an admitted lemma.
- Your latest summary distinguishes:
  - **stated** theorems/lemmas (2217),
  - **proved without recursive admits** (2124),
  - remainder are admitted or depend on admitted material.
- This is the right metric for "usable library":
  - downstream theorems are trustworthy if their dependency closure is admit-free.

Phenomenon: "self-healing" statements under checker pressure

- During iterative autoformalization, the agent sometimes **repairs** a theorem statement by adding a *missing assumption* discovered through failed proof attempts.
- Example (topology 4353 / §41.8): the agent added the hypothesis

$$C \subseteq \mathcal{P}(X)$$

  to match the intended meaning "C is a family of subsets of X".
- Consequences:
  - previously "mysterious" obligations disappear (e.g. needing $x \in X$ can be derived from $x \in c$ and $c \in C$),
  - call sites update to thread the new assumption consistently.
- Interpretation: proof checking acts as a **specification debugger**—not only for proofs, but for *statements*.

Root cause: latent domain assumptions missing from definitions

- The deeper issue is definitional: the Megalodon definition of local finiteness implicitly uses the LaTeX convention "F is a family of subsets of X" but **does not require** $F \subseteq \mathcal{P}(X)$.
- Current definition pattern:
  - assumes topology_on X T_X,
  - quantifies $A \in F$ and requires finiteness of the subcollection meeting a neighborhood,
  - but never asserts $A \subseteq X$ (or $F \subseteq \mathcal{P}(X)$).
- Practical effect: recurring friction in downstream lemmas (partition of unity, paracompactness), repeatedly requiring ad-hoc derivations/patches.
- Open question: can iterative reuse create enough "pressure" that the system **learns to strengthen** such core definitions (instead of patching at each use site)?

Mapping from the informal source

From the original formalized LaTeX file:

- Theorem: 126 Lemma: 37 Corollary: 13 (Total: 176)
- Example: 115 With Examples: 291
- Exercises: 207 (no proofs)
- Everything: 498

Marking coverage:

- About **110–120** of the 176 Thm/Lem/Cor appear clearly marked in the formalization;
- the remaining items may require additional "detective work" to align.

Dependency tracking and choke points

- Late addition in the paper: script emits, per theorem:
  - proof length, admit/recadmit flags, direct dependencies.
- Why it matters:
  - identifies **bottleneck lemmas** whose admits block many results,
  - guides the agent toward high-leverage proof completion.
- This is essentially **premise-selection for humans/agents** at the level of project management.

Major proved theorems (examples from paper)

Selected large proofs (paper lists $\geq$ 300-line proofs; all admit-free in their dependency closure):

- **Tietze Step II (real extension, nonempty)**: 10,369 lines, 142 direct deps.
- **Urysohn lemma**: 2,964 lines.
- **Urysohn metrization theorem**: 2,174 lines.
- **One-point compactification exists**: 1,719 lines.
- **Heine–Borel**: 380 lines (as a representative compactness+analysis milestone).

Parallel experiment: Urysohn Metrization in HOL Light

- Extra project: full autoformalization of the **Urysohn Metrization Theorem** in **HOL Light**.
- Motivation: a challenge posed by **John Harrison**, who already had Urysohn's lemma and extensive topology, but not metrization.
- Setup:
  - System: HOL Light (tactical proof style),
  - Agent: Claude Code,
  - Proof size: $\sim$**850 lines** for metrization alone.
  - Same infinite LLM/proof-checker loop as with Megalodon
- Time to completion: $\sim$**1 day net working time** of LLM–ITP feedback.
- Outcome: theorem completed and accepted by HOL Light; later confirmed by John Harrison.

Lessons: proof style, tooling, and agent friction

- Initial performance seemed poor: the LLM struggled with HOL Light's long **tactical scripts**.
- Improvements came from:
  - exposing the current goal during failures,
  - enforcing proof decomposition into explicit sublemmas,
  - suppressing hanging tactics (e.g. a long `MESON` prevented by a timeout).
- Observations:
  - Tactical proofs seem more brittle; harder for LLMs to debug in batch mode?
  - sophisticated tactic libraries are powerful but also harder to master?
  - declarative styles (Megalodon, Mizar-like, Naproche) appear more LLM-aligned.
  - good error reporting seems essential
- Follow-up: Stone's theorem (metrizable $\Rightarrow$ paracompact) was also autoformalized in HOL Light (**$\sim$1.6k lines, $\sim$1 day**), roughly in parallel with Megalodon.

Case study I: Urysohn lemma as an autonomy stress test

- A focused "finish this first" prompt was used to prevent drift.
- Outcome in paper: a fully finished multi-thousand-line proof by end of 2025.
- What it demonstrates:
  - agent can sustain long constructive arguments under checker pressure,
  - management constraint (no switching) can trade breadth for depth.

Case study II: "Battle of the Tietze Hill" (forward references)

- Tietze extension theorem (Munkres §35) typically relies on uniform convergence / completeness facts.
- Formalization snag: completeness machinery appears later (Munkres §43) $\Rightarrow$ forward-reference hazard.
- Paper narrative:
  - initial "unroll finite partial sums" failure blew up proof size,
  - agent later refactored/moved needed infrastructure earlier (with temporary stubs).
- This is **software-engineering behavior**: reorganize the dependency graph rather than brute-force details.

Operational lessons: sandboxing and "agent safety"

- Running the coding agent with shell access motivates isolation (bwrap).
- Observed issues (paper + podcast):
  - confusion about file locations / names,
  - occasionally destructive operations unless guarded,
  - hence strict "never throw away useful work" rules + backups.
- Pragmatic stance: treat the agent as *powerful but clumsy*.

Context limits and session "compactification"

- Long-running sessions hit internal limits (paper reports session JSONL size ceiling events).
- Mitigation in paper:
  - reset to a smaller earlier session file,
  - continue with the existing codebase as ground truth.
- General lesson:
  - persistent state should live in **code + summaries + tooling**,
  - not solely in the chat transcript.

Cost model and replication claims

- Paper headline: $\sim$ \$100 model-usage estimate for the two-week surge (Dec 22–Jan 4).
- Subscription framing: ChatGPT Pro at \$200/month enabled longer uninterrupted runs (but weekly usage limits still appear).
- For replication:
  - modest budget + automation + checker speed can dominate,
  - model choice matters, but **workflow** is the multiplier.

What the podcast emphasizes (and what experts should filter)

- Strong narrative hook: topology as the "final boss"; the speed/cost shock.
- Useful framing:
  - difference between informal proof sketches and formal proof objects,
  - "bouncer" metaphor: checker enforces zero hallucination tolerance.
- Expert caveats:
  - line counts are not directly comparable across systems,
  - admits/recadmit bookkeeping is essential to interpret progress,
  - library quality includes maintainability, naming discipline, and future refactors.

Research questions opened by the experiment

- **Generalization across assistants:** how much is checker-dependent vs loop-dependent?
- **Proof compression / refactoring:** can agents later shrink giant proofs (Tietze) into reusable abstractions?
- **Premise selection + hammering:** integrate ATP calls with caching + reconstruction to reduce low-level proof burden.
- **Curriculum inference:** can the agent learn to reorder textbook material systematically (avoid forward refs)?
- **Quality assurance:** systematic alignment between informal theorem numbering and formal objects.

Zar Goertzel's Related Work

Algebraic Foundations of Inference

From Autoformalization through Proof Repair to Theory Ex
– Z

- Knuth & Skilling's *Foundations of Inference* article aims to provide an intuitive, more general c probability theory from algebraic symmetries over *an event lattice* and a *valuation space*, impl Cox, Jaynes, and de Finetti.
- However, the proofs seemed difficult to go through and check for correctness....
- Initial *autoformalization* in Lean 4, Mathlib 4.25, with Claude Code and Codex ran into endless *"circularities"* in the proof (and Claude does make *excuses*).
- Upon request, Claude introduced *KS Separation* as a lemma, which it tried to prove, and event counterexample proving it necessary (after Ben Goertzel suggested axiomatizing it).
- Codex suggested a *direct cuts*-based proof, which is much simpler than the one in the paper.
- And then GPT 5.2 Pro dug up Alimov's work in 1950 on deriving commutativity in ordered ser without "anomalous pairs" (essentially infinitesimals), further simplifying the development, a combined with Hölder's embedding has been formalized in Lean already.

Zar Goertzel's Related Work

AI-driven Algebraic Foundations of Infere

- The AIs found alternative proofs for the other main results of KS's paper, too.
- The MO of requesting *"counterexamples"* motivating certain axioms as necessary seems hel
- By the end, they found *completeness and Scott continuity* conditions over the event lattice a that lead to this KS set-up extending to σ-Additivity, i.e., conjecturing and theory exploration.
- In summary, the era of autoformalization seems to be arriving. QED soon.

The example necessitating the separation property

```
-- Type definitions (lines 38-40)
abbrev SDBase := PN ×ₗ N
abbrev SD := WithBot SDBase

-- Semidirect operation: (u,x) ⊕ (v,y) = (u+v, x + 2^u·y) (lines 52-59)
def baseOp (p q : SDBase) : SDBase :=
  (p.1 + q.1, p.2 + (N.pow 2 (p.1 : N)) * q.2)
def op : SD → SD → SD
  | ⊥, b => b | a, ⊥ => a | some p, some q => some (baseOp p q)

-- All K&S base axioms satisfied (lines 315-324)
instance : KnuthSkillingAlgebra SD where
  op := op; ident := ⊥
  op_assoc := op_assoc; op_ident_right := ...; op_ident_left := ...
  op_strictMono_left := ...; op_strictMono_right := ...; ident_le := ...

-- Key theorems: non-commutative, fails separation (lines 333, 447-449)
theorem op_not_comm : op exX exY ≠ op exY exX := by simp; decide
theorem not_KSSeparation : ¬ KSSeparation SD := ...
```

Acknowledgments / coworkers (from the paper)

- Chad Brown: Megalodon, hammer, foundational library; setup help and feedback.
- Zar Goertzel: continuous exchange about coding agents; impetus for switching to ChatGPT 5.2; related Lean experiments.
- Atle Hahn, Adrian De Lon: related work with LLM-based autoformalization into Naproche; strong prompting examples.
- John Harrison, Larry Paulson, Dominic Mulligan: encouragement/support.
- Cezary Kaliszyk: Megalodon/hammer infrastructure; early learning-based autoformalization efforts since 2014.

Conclusion: why experts should care

- The experiment suggests a near-term regime where:
  - **scale** comes from persistent loops + cheap iteration,
  - **difficulty** shifts from proof search to *project governance* and dependency control,
  - **human role** becomes architect/editor of a growing verified corpus.
- Your latest numbers (bck5016; 2124 recadmit-free theorems/lemmas) indicate the approach continues to scale beyond the paper snapshot.